

Sicherheitsbericht

Audity — Streamline the Audit Process

Erstellt am 23.6.2026, 09:57:00 · Branch main · vertraulich

Umfang

Code-gestützte Sicherheitsanalyse der lokalen Installation (Branch main). Schwerpunkte: Authentifizierung, Mandanten-/Zugriffskontrolle, Token-Handling, SQL-Injection, Secrets-Management, CSRF/XSS und Datenintegrität.

Zusammenfassung



2 Befunde wurden im Zuge dieser Analyse

Befunde

F-01 · HOCH · Behoben

Cross-Tenant IDOR in den Framework-Suggestion-Routen

apps/api/src/cockpit/transitions.ts

Beschreibung

Die Routen GET /api/customers/:id/framework-suggestions und POST /api/customers/:id/framework-suggestions/:frameworkId/deprecate operierten auf customer_id = :id, prüften aber nur die Permission (assessment.view / assessment.edit) und nie canAccessCustomer. In dieser Multi-Tenant-App konnte dadurch jeder Nutzer mit der Permission fremde Kunden-Frameworks auslesen bzw. per deprecate verändern — ein horizontaler Berechtigungsdurchbruch (IDOR).

Auswirkung

Lesen- und Schreibzugriff auf Datensätze fremder Mandanten (Vertraulichkeit & Integrität).

Maßnahme

Beide Routen erhalten vor der DB-Operation den canAccessCustomer-Check (404 bei fehlendem Zugriff) — konsistent zu allen übrigen /api/customers/:id-Routen.

F-02 · MITTEL · Behoben

TOCTOU-Race im Customer-Acknowledgement-Redeem

apps/api/src/customerAck/routes.ts, tokens.ts

Beschreibung

Zwischen der Statusprüfung (status !== 'pending') und markTokenRedeemed lag ein Time-of-check/Time-of-use-Fenster. markTokenRedeemed aktualisierte das Token bedingungslos, sodass zwei gleichzeitige Redeem-Requests (doppelter Submit-Klick / Retry) beide den Gate passierten und je einen audit_signoffs-Eintrag erzeugten.

Auswirkung

Doppelte / inkonsistente Kunden-Sign-offs für ein einzelnes Token (Datenintegrität, Beweiskraft des Audit-Trails).

Maßnahme

markTokenRedeemed claimt das Token jetzt atomar per guarded UPDATE (redeemed_at is null AND revoked_at is null AND expires_at > now()) und gibt zurück, ob es gewonnen hat. Die Route claimt VOR dem Insert und bricht mit HTTP 410 ab, wenn sie das Race verliert.

O-01 · INFO · Beobachtung (offen)

Hash-Chain des Activity-Logs nicht serialisiert

apps/api/src/customerAck/expiryJob.ts

Beschreibung

Der Expiry-Job liest den prev_hash des letzten Log-Eintrags und schreibt anschließend einen neuen Eintrag — nicht atomar. Unter gleichzeitigen Schreibvorgängen aus anderen Quellen kann die manipulationssichere Hash-Kette verzweigen.

Auswirkung

Geringe Wahrscheinlichkeit; betrifft die Lückenlosigkeit der Tamper-Evidence-Kette, nicht die Vertraulichkeit.

Maßnahme

Empfehlung: Activity-Log-Appends serialisieren (Postgres Advisory Lock oder einzelner Writer), damit prev_hash !' event_hash konfliktfrei verkettet.

Positiv bewertete Kontrollen

- ' Passwort-Hashing durchgängig mit argon2id (Login, Passwortwechsel, MFA-Recovery-Codes).
- ' Admin-seitig generierte Einmalpasswörter: 24 Zeichen, garantierte Komplexität, CSPRNG.
- ' JWT signiert mit App-Secret; Auth-Cookies httpOnly + SameSite=strict.
- ' CSRF-Token-Validierung auf allen mutierenden Routen (requireCsrfPermission).
- ' Redis-gestütztes Rate-Limiting auf Auth-Endpunkten und Portal-Routen.
- ' Durchgängig parametrisierte SQL-Queries — keine Injection-Vektoren gefunden.
- ' Keine eval/child_process/exec; kein dangerouslySetInnerHTML/innerHTML im Frontend.
- ' Keine hartkodierten Secrets; Produktions-Config verweigert Start bei Default-/schwachen Secrets (<32 Zeichen) und erzwingt getrennten Encryption-Key.
- ' Pinned-Snapshot + event_hash sichern Berichtsstand und Sign-off-Nachweise revisionssicher.